

2-1

리스트

리스트의 개념 및 생성 방법

파이썬에서 리스트(list)는 여러개 의 값을 하나로 묶어서 저장하는 자료형

- 생성 방법: 리스트를 만들 때는 대괄호([])로 감싸주고 각 요솟값은 쉼표(,)로 구분해 준다.

```
리스트명 = [요소1, 요소2, 요소3, ...]
```

PYTHON

- 리스트의 생김새

```
a = []  
b = [1, 2, 3]  
c = ['life', 'is', 'very', 'beautiful']  
d = [1, 2, 'life', 'is']  
e = [1, 2, ['life', 'is']]
```

PYTHON

- 리스트에는 어떠한 자료형도 포함 할 수 있다.
- 리스트 저장방법

```
a = [0, 0, 0]  
a[0] = 3  
a[1] = 4  
a[2] = 5  
print(a)
```

PYTHON

- 리스트 사용 방법

각 변수 사용	리스트 사용
a, b, c, d = 10, 20 30, 40	aa = [10, 20 ,30 ,40]
a 사용	aa[0] 사용
b 사용	aa[1] 사용
c 사용	aa[2] 사용

각 변수 사용

d 사용

리스트 사용

aa[3] 사용

- 리스트는 각 인덱스의 시작이 0이다.

빈 리스트의 생성과 항목 추가

- append 매서드를 사용한 항목 추가

```
aa = []
aa.append(0)
aa.append(0)
aa.append(0)
print(aa)
```

PYTHON

출력결과
[0,0,0,0]

- for 문을 활용한 항목 추가

```
aa = []
for i in range(0, 100):
    aa.append(0)
len(aa)
```

PYTHON

출력 결과
100

> len()은 리스트의 항목의 개수를 세는 함수

리스트 값에 접근하는 다양한 방법

1. 음수값으로 접근

```
aa = [10, 20, 30, 40]

print("aa[-1]은 %d, aa[-2]는 %d" % (aa[-1], aa[-2]))
```

SHELL

출력

aa[-1]은 40, aa[-2]는 30

- 음수값으로 접근하면 뒤 항목부터 접근한다. aa는 aa[-4]까지 접근되므로 aa[-5]는 오류를 발생한다.

2. 리스트에 접근할 때 콜론(:)을 사용해 범위를 지정

PYTHON

```
aa = [10, 20, 30, 40]
aa[0:3]
aa[2:4]
```

출력 결과

```
[10, 20, 30]
[30, 40]
```

3. 콜론의 앞이나 숫자 생략

PYTHON

```
aa = [10, 20, 30, 40]
aa[2:]
aa[:2]
```

출력 결과

```
[30, 40]
[10, 20]
```

4. 리스트끼리 덧셈, 곱셈 연산

PYTHON

```
aa = [10, 20, 30]
bb = [40, 50, 60]
aa + bb
aa * 3
```

출력 결과

```
[10, 20, 30, 40, 50, 60]
[10, 20, 30, 10, 20, 30, 10, 20, 30]
```

5. 리스트의 항목 건너뛰며 추출

PYTHON

```
aa = [1, 2, 3, 4, 5, 6, 7]
aa = [::2]
aa = [::-2]
aa = [::-1]
```

출력 결과

```
[1, 3, 5, 7]
[7, 5, 3, 2]
[7, 6, 5, 4, 3, 2, 1]
```

리스트 값의 변경

1. 두 번째에 위치한 값을 1개 변경하는 방법

PYTHON

```
aa = [1, 2, 3]
a[1] = 200
a
```

출력 결과

```
[1, 200, 3]
```

2. 두 번째 값인 2를 200과 300이라는 2개의 값으로 변경

PYTHON

```
a = [1, 2, 3]
a[1:2] = [200, 300]
a
```

출력 결과

```
[1, 200, 300, 3]
```

3. 그냥 a[1] 사용시

PYTHON

```
a = [1, 2, 3]
a[1] = [200, 300]
a
```

출력 결과

[1, [200, 300], 3]

4. 리스트 값 삭제

PYTHON

```
a = [1, 2, 3, 4, 5]; del(a[1]); a  
a = [1, 2, 3, 4, 5]; a[1:3] = []; a
```

출력 결과

[1, 3, 4, 5]

[1, 5]

5. 리스트 자체 삭제

PYTHON

```
a = [1, 2, 3]; a = []; a  
a = [1, 2, 3]; a = None; a  
a = [1, 2, 3]; del(a); a
```

출력 결과

[]

아무것도 안나옴

오류 발생

리스트 조작 함수

함수	설명	사용법
append()	리스트 맨 뒤에 항목을 추가한다.	리스트명.append(값)
pop()	리스트 맨 뒤의 항목을 빼낸다. (리스트에서 해당 항목이 삭제된다)	리스트명.pop()
sort()	리스트의 항목을 정렬한다.	리스트명.sort()
reverse()	리스트 항목의 순서를 역순으로 만든다.	리스트명.reverse()
index()	지정한 값을 찾아 해당 위치를 반환한다.	리스트명.index(찾을값)
insert()	지정된 위치에 값을 삽입한다.	리스트명.insert(위치, 값)
remove()	리스트에서 지정한 값을 삭제한다. 단, 값이 여러 개면 첫 번째 값만 지운다.	리스트명.remove(지울값)
extend()	리스트 뒤에 리스트를 추가한다. 리스트의 더하기(+) 연산과 기능이 동일하다.	리스트명.extend(추가할리스트)

함수	설명	사용법
count()	리스트에서 해당 값의 개수를 센다.	리스트명.count(찾을값)
clear()	리스트의 내용을 모두 지운다.	리스트명.clear()
del()	리스트에서 해당 위치의 항목을 삭제한다.	del(리스트명[위치])
len()	리스트에 포함된 전체 항목의 개수를 센다.	len(리스트명)
copy()	리스트의 내용을 새로운 리스트에 복사한다.	새리스트 = 리스트명.copy()
sorted()	리스트의 항목을 정렬해서 새로운 리스트에 대입한다.	새리스트 = sorted(리스트)

2차원 리스트

1차원 리스트를 여러 개 연결한 것, 첨자를 2개 사용한다.

1. 예시

```
#1차원
a = [1,2,3]
#2차원 리스트
a = [[1,2,3,4],
      [5,6,7,8],
      [9,10,11,12]]
```

PYTHON

튜플

- 순서대로 묶어서 저장하는 자료형
- 특징
 1. 변경 불가능: 한 번 만들면 값을 수정, 추가, 삭제할 수 없음-> 읽기만 가능해 읽기 전용 자료를 저장할 때 사용
 2. 순서가 있음: 인덱스로 접근 가능
 3. 서로 다른 자료형 저장 가능

튜플의 생성

1. 튜플은 소괄호 ()로 생성
 - 튜플은 소괄호() 생략 가능, 항목이 하나인 튜플은 뒤에 쉼표(,)붙임

```
t1 = (10 , 20 ,30); t1
t2 = 10, 20, 30; t2
t3 = (10,); t3
t4 = 10,; t4
```

출력

```
(10, 20, 30)
(10, 20, 30)
(10,)
(10,)
```

튜플의 사용

기본적으로 튜플 값을 바꿀수 없다는 것을 제외하면 리스트 사용법과 거의 같다.

1. 튜플 범위에 접근

```
t1 = (10 , 20 ,30, 40)
t1[1:3]
t1[1:]
t[3]
```

출력

```
(20, 30)
(20, 30, 40)
(10,20, 30)
```

2. 튜플의 덧셈 및 곱셈 연산

```
t1 = (10 , 20 )
t2 = ('A', 'B')
t1 + t2
t1*3
```

출력

```
(10, 20, 'A', 'B')
(10,20,10,20,10,20)
```

튜플-> 리스트-> 튜플 변환

PYTHON

```
my_tuple = (10, 20, 30)
my_list = list(my_tuple) #튜플을 리스트로 변환한 값을 저장
my_list.append(40)
my_tuple = tuple(my_list)
my_tuple
```

출력

(10, 20, 30, 40)

딕셔너리(dictionary)

- 키(key) : 값(value) 쌍으로 데이터를 저장하는 자료형
- 이름표(키)로 값을 찾는 구조

딕셔너리의 개념

1. 쌍 2개가 하나로 묶인 자료구조

- 중괄호 {}로 묶어 구성, 키(key)와 값(value)의 쌍으로 구성

PYTHON

딕셔너리변수 = {키: 값1, 키2:값2, 키3: 값3, ...}

딕셔너리의 생성

- 키와 값은 사용자가 지정하는 것이지 규정은 없음
- 주의점: 딕셔너리에는 순서가 없어 생성한 순서대로 딕셔너리가 구성되어 있다는 보장 없음

PYTHON

```
dic = {1:'a', 2:'b', 3:'c', 4:'d'}
dic
```

출력 결과

{1:'a', 2:'b', 3:'c', 4:'d'}

2. 딕셔너리 새로운 데이터 추가

PYTHON

```
dic = {1:'a', 2:'b', 3:'c', 4:'d'}  
dic[5] = 'e'  
print(dic)
```

출력 결과

```
{1: 'a', 2: 'b', 3: 'c', 4: 'd', 5: 'e'}
```

3. 내부 데이터 삭제

PYTHON

```
dic = {1:'a', 2:'b', 3:'c', 4:'d'}  
del(dic[2])  
print(dic)
```

출력 결과

```
{1: 'a', 3: 'c', 4: 'd'}
```

4. 동일한 키를 가진 인덱스를 저장할 때 동일한 키를 갖는 딕셔너리를 생성하는 것이 아니라 마지막에 있는 키가 적용

PYTHON

```
dic = {1:'a', 2:'b', 3:'c', 4:'d', 1 : 'f'}  
print(dic)
```

```
{1: 'f', 2: 'b', 3: 'c', 4: 'd'}
```

딕셔너리 접근

1. 키로 값에 접근

PYTHON

```
student= {'학번' : 100, '이름': '강호동', '학과': '전기전자'}  
  
print(student['학번'])  
print(student['이름'])  
print(student['학과'])
```

출력 결과

100

강호동

전기전자

2. 딕셔너리명.get(키) 함수를 사용해 키로 값에 접근

PYTHON

```
student= {'학번' : 100, '이름': '강호동', '학과': '전기전자'}  
  
print(student.get('이름'))
```

출력 결과

강호동

- 딕셔너리명[키]와 딕셔너리명.get(키)는 결과 같음
 - 딕셔너리명[키]는 없는 키를 호출할 경우 오류가 발생하지만 딕셔너리명.get(키)는 없는 키를 호출하면 아무것도 반환하지 않음
3. 딕셔너리명.keys() 함수를 사용해 키로 값에 접근 : 딕셔너리의 키 값들 출력

PYTHON

```
student= {'학번' : 100, '이름': '강호동', '학과': '전기전자'}  
  
print(student.keys())
```

출력 결과

dict_keys(['학번', '이름', '학과'])

4. 딕셔너리명.items() 함수를 사용하면 튜플 형태로도 구할 수 있음

PYTHON

```
student= {'학번' : 100, '이름': '강호동', '학과': '전기전자'}  
print(student.items())  
dict_items([('학번', 100), ('이름', '강호동'), ('학과', '전기전자')])
```

문자열

문자열의 개념

1. 리스트 코드와 비교

- 리스트는 대괄호 []로 묶고 문자열은 작은 따옴표로 묶어 출력

```
s = "안녕하세요"

print(s[0])
print(s[1:3])
print(s[3:])
```

SHELL

출력결과
안
녕하
세요

2. 더하기(+) 기호 사용해 연결, 곱하기(*) 기호 사용 문자열 반복

```
s = '안녕' + '하세요'
print(s)
s = '안녕'*3
print(s)
```

SHELL

```
>>> 안녕하세요
>>> 안녕안녕안녕
```

3. len() 함수: 리스트나 문자열의 개수를 셀 때 사용

```
s = '강원대학교'
print(len(s))
```

SHELL

```
>>> 5
```

문자열 함수 메서드

파이썬 문자열 메서드/함수 정리

1. 대/소문자 변환 메서드

메서드	기능	기본 사용법	예시	비고
<code>upper()</code>	전체를 대문자로 변환	<code>s.upper()</code>	<code>"Abc".upper() -> "ABC"</code>	:contentReference[oaicite:1]{index=1}
<code>lower()</code>	전체를 소문자로 변환	<code>s.lower()</code>	<code>"AbC".lower() -> "abc"</code>	:contentReference[oaicite:2]{index=2}
<code>swapcase()</code>	대↔소문자 서로 뒤집기	<code>s.swapcase()</code>	<code>"AbC".swapcase() -> "aBc"</code>	:contentReference[oaicite:3]{index=3}
<code>title()</code>	각 단어의 첫 글자를 대문자로 (타이틀 케이스)	<code>s.title()</code>	<code>"python is easy".title() -> "Python Is Easy"</code>	:contentReference[oaicite:4]{index=4}

2. 문자열 찾기(검색) 메서드

메서드	기능	기본 사용법	예시	비고
<code>count(x)</code>	부분 문자열 <code>x</code> 가 몇 번 나오는지	<code>s.count("a")</code>	<code>"banana".count("a") -> 3</code>	:contentReference{index=5}
<code>find(x[, start[, end]])</code>	<code>x</code> 의 첫 위치 인덱스, 없으면 <code>-1</code>	<code>s.find("a")</code>	<code>"banana".find("a") -> 1</code>	실패 시 <code>-1</code> :contentReference{index=6}
<code>rfind(x[, start[, end]])</code>	<code>x</code> 의 마지막 위치 인덱스, 없으면 <code>-1</code>	<code>s.rfind("a")</code>	<code>"banana".rfind("a") -> 5</code>	:contentReference{index=7}
<code>index(x[, start[, end]])</code>	<code>find</code> 와 같지만 없으면 예러	<code>s.index("a")</code>	<code>"banana".index("a") -> 1</code>	실패 시 <code>ValueError</code> :contentReference{index=8}

메서드	기능	기본 사용법	예시	비고
<code>rindex(x[, start[, end]])</code>	<code>rfind</code> 와 같지만 없으면 에러	<code>s.rindex("a")</code>	<code>"banana".rindex("a") -> 5</code>	실패 시 <code>ValueError: contentReference {index=9}</code>
<code>startswith(prefix)</code>	prefix로 시작하면 True	<code>s.startswith("Py")</code>	<code>"Python".startswith("Py") -> True</code>	:contentReference {index=10}
<code>endswith(suffix)</code>	suffix로 끝나면 True	<code>s.endswith("on")</code>	<code>"Python".endswith("on") -> True</code>	:contentReference {index=11}

3. 공백/문자 삭제 & 문자열 변경

메서드	기능	기본 사용법	예시	비고
<code>strip([chars])</code>	양쪽 (앞+뒤) 공백/특정문자 제거	<code>s.strip()</code> / <code>s.strip("-")</code>	<code>" a ".strip() -> "a"</code>	<code>chars</code> 는 "집합"처럼 동작 :contentReference {index=12}
<code>lstrip([chars])</code>	왼쪽만 제거	<code>s.lstrip()</code>	<code>" a".lstrip() -> "a"</code>	:contentReference {index=13}
<code>rstrip([chars])</code>	오른쪽만 제거	<code>s.rstrip()</code>	<code>"a ".rstrip() -> "a"</code>	:contentReference {index=14}
<code>replace(old, new[, count])</code>	<code>old</code> 를 <code>new</code> 로 치환	<code>s.replace("a", "b")</code>	<code>"banana".replace("a", "A") -> "bAnAnA"</code>	<code>count</code> 로 치환 횟수 제한 :contentReference {index=15}

4. 분리/결합

메서드	기능	기본 사용법	예시	비고
<code>split(sep=None)</code>	구분자 기준으로 쪼개	<code>s.split()</code> / <code>s.split(",")</code>	<code>"a,b,c".split(",") -> ["a", "b", "c"]</code>	<code>sep</code> 미지정이면 연속 공백도 리

메서드	기능	기본 사용법	예시	비고
	개서 리스트 반환			:contentReference[oaicite:16]
<code>splitlines()</code>	줄바꿈 기준으로 분리	<code>s.splitlines()</code>	<code>"a\\nb".splitlines() -> ["a", "b"]</code>	<code>keepends=True</code> 옵션도 있음 고) :contentReference[oaicite:17]
<code>join(iterable)</code>	iterable(리스트 등)의 문자열들 사이에 끼워 결합	"구분자". <code>join(list)</code>	<code>"-".join(["a", "b"]) -> "a-b"</code>	join의 주체는 "구분자 문자" :contentReference[oaicite:18]

5. 함수(Function) vs 메서드(Method) + 자주 쓰는 함수

이름	종류	기능	기본 사용법	예시	비고
<code>len(x)</code>	함수	길이(문자 개수 등) 반환	<code>len(s)</code>	<code>len("abc") -> 3</code>	메서드가 아니라 함수 호출 형태 :contentReference[oaicite:19]
<code>map(func, iterable)</code>	함수	iterable의 각 요소에 func 적용	<code>list(map(int, lst))</code>	<code>list(map(int, ["1", "2"])) -> [1, 2]</code>	결과를 보려면 list/tuple로 감싸는 경우 많음 :contentReference[oaicite:20]

6. 정렬/채우기(폭 맞추기)

메서드	기능	기본 사용법	예시	비고
<code>center(width[, fill])</code>	가운데 정렬 + 폭 맞춤	<code>s.center(10)</code>	<code>"hi".center(6, "-") -> "--hi--"</code>	fill은 1글자만 가능 :contentReference[oaicite:21]
<code>ljust(width[, fill])</code>	왼쪽 정렬	<code>s.ljust(10)</code>	<code>"hi".ljust(5, ".") -> "hi..."</code>	:contentReference[oaicite:22]
<code>rjust(width[, fill])</code>	오른쪽 정렬	<code>s.rjust(10)</code>	<code>"hi".rjust(5, ".") -> "...hi"</code>	:contentReference[oaicite:23]
<code>zfill(width)</code>	왼쪽을 0으로 채움(숫자)	<code>s.zfill(5)</code>	<code>"42".zfill(5) -> "00042"</code>	부호도 처리됨: <code>"-42".zfill(5) -> "-00042"</code>

메서드	기능	기본 사용법	예시	비고
	자 문자 열에 자 주)			:contentReference[oaicite:24] {index=24}

7. 문자열 구성(문자 종류) 판별

메서드	기능	기본 사용법	예시	비고
<code>isdigit()</code>	전부 숫자 면 <code>True</code>	<code>s.isdigit()</code>	<code>"123".isdigit() -> True</code>	공백/부호/점 포함되면 False :contentReference[oaicite:25] {index=25}
<code>isalpha()</code>	전부 문자 (알파벳/ 한글 등) 면 <code>True</code>	<code>s.isalpha()</code>	<code>"abc".isalpha() -> True</code>	숫자/공백 섞이면 False :contentReference[oaicite:26] {index=26}
<code>isalnum()</code>	전부 문자 또는 숫자 면 <code>True</code>	<code>s.isalnum()</code>	<code>"abc123".isalnum() -> True</code>	공백/특수문자 있으면 False :contentReference[oaicite:27] {index=27}
<code>islower()</code>	영문이 전 부 소문자 면 <code>True</code>	<code>s.islower()</code>	<code>"abc".islower() -> True</code>	영문이 없으면 애매(케이스 체크 필요) :contentReference[oaicite:28] {index=28}
<code>isupper()</code>	영문이 전 부 대문자 면 <code>True</code>	<code>s.isupper()</code>	<code>"ABC".isupper() -> True</code>	:contentReference[oaicite:29] {index=29}
<code>isspace()</code>	전부 공백 문자(스페 이스/탭/ 개행 등) 면 <code>True</code>	<code>s.isspace()</code>	<code>" ".isspace() -> True</code>	빈 문자열 ""은 False :contentReference[oaicite:30] {index=30}